

Práctica: Máquina de estados del flujo de aprobación

Paso a paso — validar transiciones y registrar la trazabilidad

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 3 — Modelamiento de datos del dominio
Docente	Edwin Barrientos
Duración estimada	60-90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Implementar la máquina de estados de la solicitud en la clase parcial del dominio.
- Impedir transiciones inválidas con una excepción propia de dominio.
- Asignar el nivel de aprobación según el monto mediante un servicio de aplicación.
- Definir los DTOs e implementar `SolicitudService` para orquestar las transiciones.
- Exponer y validar los endpoints REST en `SolicitudesController` comprobando respuestas HTTP 200 y 400.

2. Desarrollo paso a paso

Paso 1 — Una excepción de dominio

Un error de negocio no es lo mismo que un error técnico. Darle su propio tipo permite traducirlo a un 400 en la API.

Domain/Excepciones/ReglaDominioException.cs

```
namespace SolicitudesApp.Domain.Excepciones;

public class ReglaDominioException : Exception
{
    public ReglaDominioException(string mensaje) : base(mensaje) { }
}
```

Paso 2 — Las transiciones válidas

Agrega estos miembros a la clase parcial `Solicitud`. La tabla de transiciones en el dominio es la única fuente de verdad del flujo.

Domain/Entidades/Solicitud.Validaciones.cs

```
using SolicitudesApp.Domain.Excepciones;
using static SolicitudesApp.Domain.Entidades.EstadoSolicitud;

namespace SolicitudesApp.Domain.Entidades;

public partial class Solicitud
{
    public bool PuedePasarA(EstadoSolicitud destino) => (Estado, destino) switch
    {
        (Borrador, Enviada) => true, // el usuario la envía
        (Enviada, EnRevision) => true, // el revisor la toma
        (EnRevision, Borrador) => true, // se devuelve observada
        (EnRevision, Aprobada) => true,
        (EnRevision, Rechazada) => true,
        (Aprobada, Pagada) => true, // tesorería paga
        _ => false
    };

    public void CambiarEstado(EstadoSolicitud destino, Usuario responsable,
        string? comentario = null)
    {
        if (!PuedePasarA(destino))
            throw new ReglaDominioException(
                $"Transición inválida: No se puede pasar de '{Estado}' a '{destino}'.");

        Estado = destino;
        FechaModificacion = DateTime.UtcNow;
        ModificadoPor = responsable.Correo;

        Aprobaciones.Add(new Aprobacion
        {
            SolicitudId = Id,
            ResponsableId = responsable.Id,
            Responsable = responsable,
            EstadoResultante = destino,
            Comentario = comentario
        });
    }
}
```

La regla vive en el dominio, no en la interfaz: aunque el frontend oculte un botón, la entidad sigue rechazando la transición inválida. Es la única forma de garantizar la integridad del flujo.

Paso 3 — Asignar el aprobador por monto

El nivel de aprobación depende del monto total. Este servicio vive en la capa de aplicación.

Application/Aprobaciones/AsignadorAprobador.cs

```
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Aprobaciones;

public class AsignadorAprobador
{
    // Los niveles se recorren de menor a mayor límite
    private static readonly (string Nivel, decimal Limite)[] _niveles =
    [
        ("Jefatura", 1000m),
        ("Gerencia", 5000m),
        ("Directorio", decimal.MaxValue)
    ];

    public string Determinar(decimal monto)
    {
        if (monto <= 0)
            throw new ArgumentException("El monto debe ser mayor a cero");

        return _niveles.First(n => monto <= n.Limite).Nivel;
    }

    public bool RequiereDobleAprobacion(decimal monto) => monto > 5000m;
}
```

Paso 4 — Los DTOs del flujo de solicitud

Define en la capa de aplicación los contratos de datos desacoplados de las entidades internas del dominio.

Application/Solicitudes/Dtos.cs

```
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Solicitudes;

public record CambiarEstadoDto(
    EstadoSolicitud Destino,
    int ResponsableId,
    string? Comentario
);

public record SolicitudRespuestaDto(
    int Id,
    string Motivo,
    decimal MontoTotal,
    string Estado,
    string AprobadorRequerido,
    IEnumerable<AprobacionHistorialDto> Historial
);

public record AprobacionHistorialDto(
    string Estado,
    string? Responsable,
    string? Comentario,
    DateTime Fecha
);
```

Paso 5 — El servicio de aplicación

Crea la interfaz y la implementación de `SolicitudService` para orquestar la máquina de estados y consultar el historial.

Application/Solicitudes/SolicitudService.cs

```
using SolicitudesApp.Application.Aprobaciones;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Application.Solicitudes;

public interface ISolicitudService
{
    SolicitudRespuestaDto? ObtenerPorId(int id);
    SolicitudRespuestaDto CambiarEstado(int id, CambiarEstadoDto dto);
}

public class SolicitudService(AsignadorAprobador asignador) : ISolicitudService
{
    private static readonly List<Usuario> _usuarios =
    [
        new() { Id = 1, Nombre = "Ana Torres", Correo = "ana@empresa.com" },
        new() { Id = 2, Nombre = "Luis Vega", Correo = "luis@empresa.com" },
        new() { Id = 3, Nombre = "Rosa Quispe", Correo = "rosa@empresa.com" },
        new() { Id = 4, Nombre = "Jose Ramos", Correo = "jose@empresa.com" }
    ];

    private static readonly Solicitud _solicitud = new()
    {
        Id = 1, Motivo = "Equipamiento de TI", CreadoPor = "ana@empresa.com",
        MontoTotal = 3200m,
        Detalles = { new() { Concepto = "Laptop", Cantidad = 1, PrecioUnitario = 3200m } }
    };

    public SolicitudRespuestaDto? ObtenerPorId(int id)
    => id == _solicitud.Id ? Mapear(_solicitud) : null;

    public SolicitudRespuestaDto CambiarEstado(int id, CambiarEstadoDto dto)
    {
        if (id != _solicitud.Id)
            throw new KeyNotFoundException($"Solicitud #{id} no encontrada");

        var resp = _usuarios.FirstOrDefault(u => u.Id == dto.ResponsableId)
            ?? throw new ArgumentException($"Responsable con ID {dto.ResponsableId} no existe");

        _solicitud.CambiarEstado(dto.Destino, resp, dto.Comentario);
        return Mapear(_solicitud);
    }

    private SolicitudRespuestaDto Mapear(Solicitud s) => new(
        s.Id, s.Motivo, s.MontoTotal, s.Estado.ToString(),
        asignador.Determinar(s.MontoTotal),
        s.Aprobaciones.Select(a => new AprobacionHistorialDto(
            a.EstadoResultante.ToString(), a.Responsable?.Nombre, a.Comentario, a.Fecha)));
}
```

Paso 6 — El controlador REST delgado

El controlador solo atiende peticiones HTTP, delega a `ISolicitudService` y traduce `ReglaDominioException` a HTTP 400 Bad Request.

Controllers/SolicitudesController.cs

```
using Microsoft.AspNetCore.Mvc;
using SolicitudesApp.Application.Solicitudes;
using SolicitudesApp.Domain.Excepciones;

namespace SolicitudesApp.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class SolicitudesController(ISolicitudService service) : ControllerBase
{
    [HttpGet("{id:int}")]
    public IActionResult ObtenerPorId(int id)
    {
        var solicitud = service.ObtenerPorId(id);
        return solicitud is not null
            ? Ok(solicitud)
            : NotFound(new { mensaje = $"Solicitud #{id} no existe" });
    }

    [HttpPost("{id:int}/transicion")]
    public IActionResult CambiarEstado(int id, [FromBody] CambiarEstadoDto dto)
    {
        try
        {
            var resultado = service.CambiarEstado(id, dto);
            return Ok(new { mensaje = $"Transición a {dto.Destino} exitosa", solicitud = resultado });
        }
        catch (ReglaDominioException ex)
        {
            return BadRequest(new { error = ex.Message }); // 400 si viola regla de dominio
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { mensaje = ex.Message });
        }
    }
}
```

El controlador no valida reglas de negocio: solo orquesta la petición HTTP y delega en el servicio. Si el dominio rechaza la transición lanzando `ReglaDominioException`, el controlador la traduce a un código HTTP 400 Bad Request estándar.

Paso 7 — Inyección de dependencias en Program.cs

Registra los controladores, Swagger y los servicios de aplicación en el contenedor de dependencias.

Program.cs

```
using SolicitudesApp.Application.Aprobaciones;
using SolicitudesApp.Application.Solicitudes;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddSingleton<AsignadorAprobador>();
builder.Services.AddSingleton<ISolicitudService, SolicitudService>();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.MapControllers();
app.Run();
```

Paso 8 — Probar el ciclo de vida y las reglas en Swagger / HTTP

Ejecuta dotnet run en SolicitudesApp.Api y prueba el camino feliz y la respuesta 400 ante saltos prohibidos desde el archivo HTTP o Swagger UI.

SolicitudesApp.Api.http

```
@baseUrl = https://localhost:7001

### 1. Consultar estado inicial
GET {{baseUrl}}/api/solicitudes/1

### 2. Flujo feliz: Enviar a revisión (Ana, ID 1) -> HTTP 200 OK
POST {{baseUrl}}/api/solicitudes/1/transicion
Content-Type: application/json

{ "destino": 2, "responsableId": 1, "comentario": "Envío para revisión" }

### 3. Revisor toma la solicitud -> HTTP 200 OK
POST {{baseUrl}}/api/solicitudes/1/transicion
Content-Type: application/json

{ "destino": 3, "responsableId": 2, "comentario": "Sustentos conformes" }

### 4. Gerencia aprueba la solicitud -> HTTP 200 OK
POST {{baseUrl}}/api/solicitudes/1/transicion
Content-Type: application/json

{ "destino": 4, "responsableId": 3, "comentario": "Autorizado por gerencia" }

### 5. Salto inválido rechazado por el dominio -> HTTP 400 Bad Request
POST {{baseUrl}}/api/solicitudes/1/transicion
Content-Type: application/json

{ "destino": 2, "responsableId": 1, "comentario": "Intentar volver a Enviada" }
```

Rechazada y Pagada son estados finales: la tabla de transiciones no define salidas desde ellos. En la API REST, cualquier intento de enviar una petición POST para cambiar de estado devolverá HTTP 400 Bad Request garantizando la inmutabilidad del flujo.

3. Retos adicionales

1. Agrega la validación de que solo un usuario con el rol Tesorería puede pasar una solicitud a Pagada en el endpoint.
2. Implementa un filtro o middleware global de excepciones para traducir `ReglaDominioException` a `ProblemDetails` (RFC 7807) con status 400.
3. Agrega un endpoint GET `/api/solicitudes/{id}/transiciones-permitidas` que devuelva los estados a los que puede avanzar la solicitud.
4. Implementa la doble aprobación cuando el monto supera los 5000: requiere dos registros de Aprobación antes de pasar a Aprobada.
5. Escribe pruebas unitarias con xUnit para las transiciones válidas e inválidas de la máquina de estados.

4. Entregables

- El controlador `SolicitudesController` expone endpoints GET y POST para consultar y transicionar solicitudes.
- El flujo feliz de Borrador a Pagada se ejecuta vía peticiones HTTP respondiendo 200 OK con el historial de auditoría.
- El intento de transición inválida responde HTTP 400 Bad Request con el mensaje de `ReglaDominioException`.
- El camino de rechazo funciona y verifica por API que Rechazada y Pagada son estados finales.
- Capturas de Swagger o del cliente HTTP demostrando las transiciones exitosas y el rechazo de saltos inválidos.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 3 · Docente: Edwin Barrientos · New Horizons